

IT Workshop

Course Code: CS2204

Lab

Dr. Rahul Mishra

IIT Patna



* USING MULTIPLE INSTRUCTIONS (-e and -f)

* The -e option allows you to enter as many instructions as you wish, each preceded by the option

* We can repeat the command prior to the previous one with this:

```
$ sed -n -e '1,2p' -e '7,9p' -e '$p' emp.lst
```

* When you have too many instructions to use or when you have a set of common instructions that you execute often, they are better stored in a file

```
$ cat instr.fil
```

* -f option to direct sed to take its instructions from the file using the command

```
$ sed -n -f instr.fil emp.lst.
```


* sed is quite liberal in that it allows a great deal of freedom in using and repeating options.

* We can use the `-f` option with multiple files:-

```
$ sed -n -f instr.fil1 -f instr.fil2 emp.lst
```

```
$ sed -n -e '/saxena/p' -f instr.fil1 -f instr.fil2 emp?.lst
```

⇒ CONTEXT ADDRESSING :-

When you specify a single pattern, all lines containing the pattern are selected.

```
→ $ sed -n '/director/p' emp.lst
```

```
→ $ sed -n '/dasgupta/, /saksena/p' emp.lst
```

```
→ $ sed -n '1, /dasgupta/p' emp.lst
```


* Using Regular Expression :-

```
$ sed -n '/[aA]gg*[an][an]wol/p' emp.lst  
$ sed -n '/sa[kx]s*ena/p  
/gupta/p' emp.lst
```

} All the agaswals
Either the saxenas
or gupta

```
$ sed -n '/50.....$/p' emp.lst
```

* Writing Selected Lines to a file (w):-

```
$ sed -n '/director/w dlist' emp.lst
```

w(write) command to write
the selected lines to a sepearate
file.

Access more than one file:-

```
$ sed -n '/director/w dlist  
/manager/w mlist  
/executive/w elist' emp.lst
```

```
$ sed -n '1,500w foo1  
501,$w foo2' foo.main
```

Saves first 500 lines in foo1
and the rest in foo2

Create any file with 1000 lines

Text Editing:

-i (insert), a (append), c (change) and d (delete)

```
$ sed 'i\
```

```
> #include <stdio.h>\
```

```
> #include <unistd.h>
```

```
>' foo.c > $$
```

} need to use `\` before [Enter] here...
and here only

- No `\` in last line of input
- Redirect output to a temporary file.

* Now, we must move this file to `foo.c` to be able to use it:

```
$ mv $$ foo.c ; head -n 2 foo.c
```



```
* sed 'a\
    > emp.lst
```

} Insert after every line
this blank line.

* Deleting lines

```
$ sed '/director/d' emp.lst > olist
```

-w option not to be used
with d.

```
$ sed -w '/director/!p' emp.lst > olist
```

select all lines except those
containing director

* Deleting Blank lines

```
sed '/^[space tab]*$/d' foo
```

buttons

* Substitution (s)

[address] s /expression 1 /expression 2 /flags

\$ sed 's/1/:/' emp.lst | head -n 2 # just the first (left most) instance of the 1 in a line has been replaced.

g (global)

\$ sed 's/1/:/g' emp.lst | head -n 2

\$ sed '1,3s/1/:/g' emp.lst # First three lines only

* sed also uses regular expressions for patterns to be substituted.

```
$ sed 's/[Aa]gg*[an][an]wal/Aganwal/g' emp.lst
```

```
$ sed 's/^/2/' emp.lst | head -n 1
```

```
$ sed 's/$/1.00/' emp.lst | head -n 1
```

Note: When a **g** is used at the end of a substitution instruction, the change is performed globally along the line.

* Performing Multiple Substitutions:

```
$ sed 's/<I>/<EM>/g
```

```
> s/<B>/<STRONG>/g
```

```
> s/<U>/<EM>/g' form.html
```


Basic File Attribute :-

[1]

* The UNIX file systems let users access other files not belonging to them and without infringing on security.

* A file also has a number of attributes (properties) that are stored in the inode.

* ls -l : Listing File Attributes =>

* -l (long) option that reveals most.

* This option displays most attributes of a file - like its permissions, size and ownership details.

* Let us use ls -l to list seven attributes of all files in the current directories.

\$ ls -l

total 72

-rw-r--r--	1	Kumar	metal	19514	May 10	13:45	chap01
-rw-r--r--	1	Kumar	metal	4174	May 10	15:01	chap02
-rw-rw-r--	1	Kumar	metal	84	Feb 12	12:30	dept.lst
drwxr-xr-x	2	Kumar	metal	512	May 9	10:31	helpdir
drwxr-xr-x	2	Kumar	metal	512	May 9	09:57	progs

* The list is preceded by the words total 72, which indicates that a total of 72 blocks are occupied by these files on disk, each block consisting of 512 bytes (1024 in Linux).

* File Type and Permissions :-

- ① first column shows the type and permission associated with each file.
- ② The first character in this column is mostly a -, which indicates that the file is an ordinary one (d for directory)

* The values r, w, x and $-$. In the UNIX system, a file can have three types of permissions $\rightarrow$ read, write, and execute. [3]

Links:

- * The second column indicates the number of links associated with the file.
 - $\hookrightarrow$ This is actually the number of file names maintained by the system of that file.
- * A link count greater than one indicates that the file has more than one name.

Ownership: • When you create a file, you automatically become its owner.

- The third column shows `kumar` as the owner of all of these files.
- The owner has full authority to tamper with a file's contents and permissions - a privilege not available with others except the root user.
- Similarly, you can create, modify or remove files in a directory if you are the owner of the directory.

Group Ownership:-

- When opening a user account, the system administrator also assigns the user to some group.
- The fourth column represents the group owner of the file. # group members often work on same file (quite popular nowadays)

File Size :- (5th column)

- The amount of data it contains.
- It is measure of only a character count of the file and not a measure of disk space that it occupies.
- The two directories show smaller file sizes (512 bytes each). # As a directory maintains a list of filenames along with an identification number.

last Modified Time

Filename.

* Listing Directory Attributes :-

[5]

\$ ls -ld helpdir prnpgs

drwxr-xr-x 2 Kumar metal 512 May 9 10:31 helpdir.

* File Ownership :-

- ① Owner
- ② Group owner
- ③ Other user

> When the system administrator creates a user account, he/she has to assign these parameters to the user:

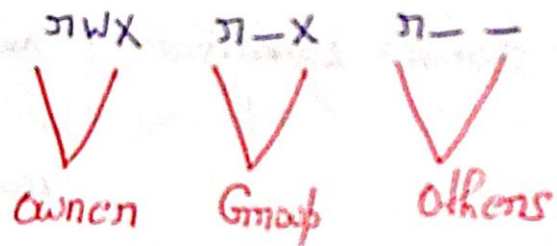
- User-id (UID) - both its name and numeric representations
- Group-id (GID) - both its name and numeric representation.

* \$ id

uid = 655537 (kumar) gid = 655535 (matal)

* File Permissions :-

> UNIX has a simple and well-defined system of assigning permissions to file.



* chmod: changing File Permissions

* owner will refer as user.

```
$ cat /usr/bin/startx > xstart
$ ls -l xstart.
```


* chmod (change mode) command is used to set the permissions of one or more files for all three categories of users (user, group, and others)

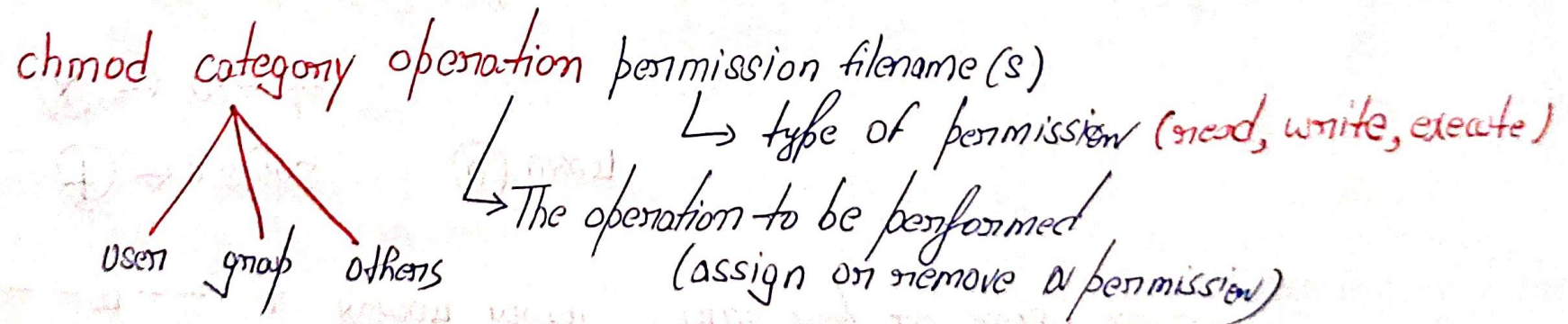
* It can be run only by the user (the owner) and the superuser.

* The command can be used in two ways:

- In a relative manner by specifying the changes to the current permissions.
- In an absolute manner by specifying the final permissions.

* Relative Permissions →

— chmod only changes the permissions specified in the command line and leaves the other permissions unchanged. →



\$ chmod (u+x) xstart

\$ ls -l xstart

-rwxr-xr-x -- -- 1 kuman metal

1906 May 10 20:30 xstart

(+) → assigns

(u) user

(x) → execute

ugo → user/group/other

\$ chmod 761 xstart

* The expression 777 signifies all permissions for all categories, while 000 indicates absence of all permissions for all categories.

777

read - 1

write - 2

exclusive - 4

761


```
chmod u-rw, go-r xstart
chmod 000 xstart
```

----- 1 kumar metal 1906 May 10 20:30 xstart

```
chmod a+rwx xstart
chmod 777 xstart
```

The resulting permissions setting is simply dangerous:

---rwxrwxrwx 1 kumar metal 1906 May 10 20:30 xstart

Using chmod Recursively (-R) :

It is possible to make chmod descend a directory hierarchy and apply the expression to every file and subdirectory it finds.

chmod -R a+x shell-scripts.

*
chmod -R 755 . # works on hidden files also
chmod -R a+x * # leaves out hidden files.

Similarly,

* changing File ownership →

chown options owner [:group] file(s)

ex → chown sharma note

* chgrp: changing Group Owner →

chgrp dba dept.lst.